



A study of efficient concurrent integration methods of B-Spline basis functions in IGA-FEM

Maciej Woźniak^{a,*}, Anna Szyszka^a, Sergio Rojas^b

^a AGH University of Sciences and Technology, Institute of Computer Science, Electronics and Telecommunication, Department of Computer Science, al. A Mickiewicza 30, 30-059 Kraków, Poland

^b Instituto de Matemáticas, Pontificia Universidad Católica de Valparaíso, Valparaíso, Chile

ARTICLE INFO

Keywords:

Isogeometric finite element method
Numerical integration
Trace theory
Sum factorization

ABSTRACT

Based on trace theory, we study efficient methods for concurrent integration of B-spline basis functions in IGA-FEM. We motivate the discussion with the heat-equation problem discretized with the forward Euler time-marching scheme in time and the IGA-FEM in space, leading to the study of the cost associated with the assembling of the Mass matrix. We consider several scenarios of parallelization for two standard integration methods; the classical one and sum factorization. We aim to efficiently utilize hybrid memory machines, such as modern clusters, by focusing on the non-obvious layer of the shared memory part of concurrency. We estimate the performance of computations on a GPU and provide a strategy for performing such computations in practical implementations.

1. Introduction

The great success of the finite element method (FEM) can be attributed to its solid theoretical rooting in the fields of variational calculus, and functional analysis [1,2]. It is widely used for numerically solving partial differential equations (PDEs) in Computer-Aided Engineering (CAE) systems. Most FEM computations consist of two phases; (a) a transformation of the PDE to a discrete form by mapping onto the finite-dimensional approximation space, (b) solving the resulting system of linear or nonlinear algebraic equations [3]. Commonly, the FEM implementation is made by computing local integral subroutines by elements, defining local element matrices that are subsequently integrated and assembled in the global system of matrix equations.

A current hot topic regarding numerical FEM approximations is the IsoGeometric Analysis FEM (IGA-FEM) [4]. It integrates the geometrical modeling of Computer Aided Design (CAD) systems with engineering computations of CAE systems.

IGA-FEM computations share the same structure as the traditional FEM. However, the main difference is that IGA-FEM employs B-splines basis functions for spanning the approximation space [5].

In several scenarios, mainly when dealing with time-dependency or non-linearities, it is well known that FEM can give rise to the resolution of a high-cost computational problem. For instance, one of the standard techniques for numerically solving time-dependent PDEs is to perform a finite difference method (FDM) in time, coupled with a FEM discretization in space. Last implies assembling multiple FEM

matrices at every time step in several scenarios. Indeed, for nonlinear PDEs, it may be required to integrate and assemble FEM matrices at each iteration step of the nonlinear solver. In particular, if an implicit method in time is employed [6–8]. Furthermore, the cost of the assembling grows with the space dimension [9]. Therefore, the cost associated with the integration in assembling FEM matrices is critical in terms of computation time.

Traditionally, the integration procedure is performed in parallel, element-by-element, making a level of concurrent operations in which data is independent. However, in [10] is proposed a methodology based on adding two levels of parallelism within each element that distinguishes the independent operations. The goal was to reduce the computational time in the integration procedure using the modern parallel architectures of a GPU [11]. Multiple existing research papers aim at optimal GPU utilization for integration in IGA-FEM [12–14]. However, the main focus is on software design level.

The paper aims to compare the practical concurrent implementation performance of the classical integration method and sum factorization with different parallelization schemes. To do that, we apply the methodology presented in [10] to sum factorization. The main conclusions are: (1) the sum factorization does not parallelize as expected, and (2) the classical integration running in GPU can be faster than sum factorization. Before describing the methodology, we briefly describe the principal concepts involved in the case study.

* Corresponding author.

E-mail address: macwozni@agh.edu.pl (M. Woźniak).

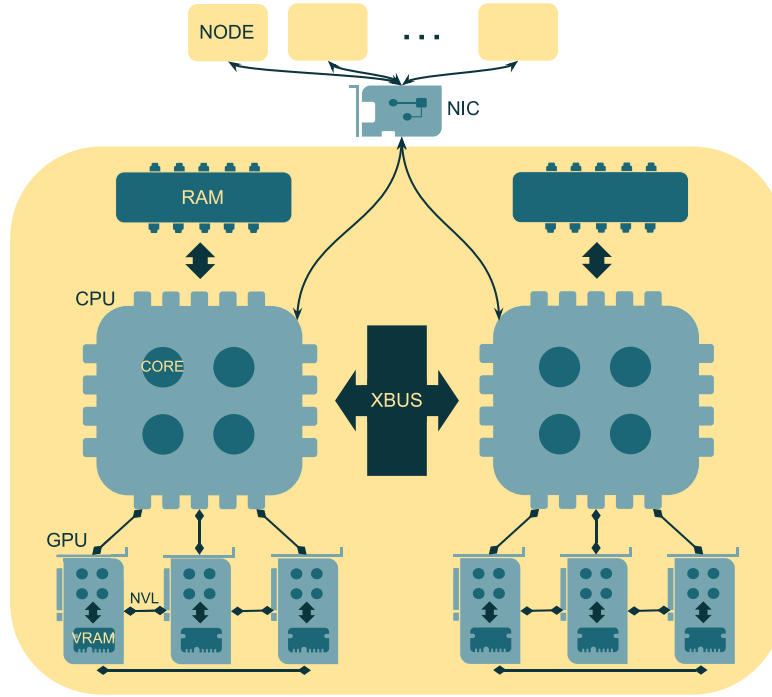


Fig. 1. Architecture of a single node of a modern supercomputer. Figure based on Summit [17] supercomputer.

1.1. Architecture

State-of-the-art supercomputers are designed as multi-level hierarchical hybrid systems [15–17]. A representative architecture is shown in Fig. 1. They consist of classical nodes (servers) communicating over a network (specialized solutions, such as Infiniband) through a Message Passing Interface (MPI). Inside every server (node), multiple multi-core CPUs partially share RAM. Furthermore, these systems have a massively parallel co-processor such as GPUs. GPUs have dedicated memory, with a hierarchical memory organization [18], which is not shared with the CPU. Concurrent algorithms dedicated to such systems are crucial for efficient hardware utilization and reduced carbon trace (green computing).

1.2. Sum factorization

Sum factorization (see, e.g. [19]) was first introduced in [20]. It was initially employed for the standard higher-order finite element method [21]. However, currently it is preferred the technique of choice for efficient formation of local element matrices in hp-finite elements [22–25] and IGA with higher-order B-splines [19,26,27]. In essence, sum factorization is a reordering of the computations in such a way as to exploit the underlying tensor product of the test and trial spaces involved. When employed, the cost of integration is reduced from $\mathcal{O}(m^3 n^3 q^3)$ to $\mathcal{O}(m^3 n^3 q + m^2 n^2 q + mnq^3)$, where m denotes the number of test functions over an element, n is the number of trial functions over the element, and q is the number of quadrature points over the element in each spatial direction. Last in practice implies that, for a given polynomial degree p , the total reduction is from $\mathcal{O}(p^9)$ to $\mathcal{O}(p^7)$ when considering Gaussian quadrature, and down up to $\mathcal{O}(p^6)$ for weighted quadrature.

1.3. Trace theory

There exist multiple methods used in the formal verification of concurrent computations. One of the most popular is the *Trace Theory* [28]. Other methods include *Petri Net* [29], *Process Calculi* [30] and *Actor Model* [31].

Trace Theory delivers the Foata Normal Form (FNF) [32] and Diekert dependency graphs, which help characterize the processing in a single element and simplify the parallel implementation on massively parallel machines, such as GPUs. Finally, it makes a base for near-optimal scheduling. It simplifies concurrent implementation on a GPU while providing a theoretical framework for verifying the correctness of such a parallel algorithm.

1.4. Structure of the article

The rest of the article is organized as follows. First, in Section 2 we describe the model problem, together with its discretization in time and space, used for the benchmarks. Next, in Section 3 we discuss the integration algorithms and apply trace theory to create a concurrent algorithm performing sum factorization. In Section 4 we consider several numerical experiments to show and discuss the performance of the integration methodologies. Finally, we conclude the paper in Section 5.

2. Model problem and IGA-discrete variational formulation

The main objective of this work is to compare the computational cost, using the classical approach versus sum factorization, when performing concurrent computations for the integration procedure in assembling IGA matrices. Therefore, in the spirit of presenting the proposed methodology in a simple setting (i.e., the extension [10] to the concurrent sum factorization algorithm), we will consider as a motivational problem the heat equation discretized in time with the forward Euler method, to simplify the discussion to the cost associated to the assemble of the Mass matrix only. However, in more general cases or scenarios, for instance, when considering more sophisticated partial differential equations, another kind of boundary conditions, or another time-marching scheme, the obtained conclusions must remain the same.

2.1. Model problem

As motivational problem, we consider the following heat-transfer model problem:

$$\begin{cases} \frac{\partial u}{\partial t} = \Delta u & \text{in } \Omega \times [0, T] \\ \nabla u \cdot \hat{\mathbf{n}} = 0 & \text{on } \partial\Omega \times [0, T] \\ u = u_0, \text{ at } t = 0 & \text{in } \Omega \end{cases} \quad (1)$$

where $\Omega = \omega = (0, 1)^3 \subset \mathbb{R}^3$ denotes the spatial domain, $\hat{\mathbf{n}}$ denotes the normal vector to the domain boundary $\partial\Omega$, $T > 0$ is a length of the time interval, and u_0 is a given initial state.

2.2. Discretization in time

To obtain a fully-discrete formulation of problem (1), we start by considering its corresponding continuous weak formulation in space, given as follows:

Find $u \in C^1((0, T), H^1(\Omega))$ such that $u = u_0$ at $t = 0$ and, for each $t \in (0, T)$, it holds:

$$\int_{\Omega} \frac{\partial u}{\partial t} v \, dx = - \int_{\Omega} \nabla u \cdot \nabla v \, dx, \quad \forall v \in H^1(\Omega). \quad (2)$$

For simplicity, we consider a discrete-in-time version of problem (2) by employing the forward Euler method. This is, denoting by u_n the approximation of u at time $t = n\Delta_t$, with $n = 0, \dots, N$ where $\Delta_t = T/N$ denotes a fixed time step for a given integer $N > 0$, we obtain $u_{n+1} \in H^1(\Omega)$ as the solution of the following variational problem:

$$\int_{\Omega} u_{n+1} v \, dx = \int_{\Omega} u_n v \, dx - \Delta_t \int_{\Omega} \nabla u_n \cdot \nabla v \, dx, \quad \forall v \in H^1(\Omega). \quad (3)$$

2.3. B-splines basis functions

A B-spline is a convenient function representing polynomial splines (see e.g. [33,34]). B-splines are characterized by the polynomial degree inside the respective elements and their regularity at the interfaces between them of the finite element mesh. For simplicity, in this work, we will consider 3D-tensor B-splines basis functions of the same polynomial degree and regularity at the interior faces of the tensor mesh. However, the methodology can be easily extended to more general B-spline basis functions.

Consider a partitioning of $\bar{\omega} = [0, 1]$ into K uniform elements $[\hat{x}_{k-1}, \hat{x}_k]$, with

$$\hat{x}_0 = 0 < \hat{x}_1 < \dots < \hat{x}_{K-1} < \hat{x}_K = 1.$$

For a given $p > 0$, the B-spline basis functions being piece-wise polynomials of degree p , with C^{p-1} regularity at the interior knots $\{\hat{x}_k\}_{k=1}^{K-1}$, are defined through the following knot vector:

$$\Xi = \{\gamma_i\}_{i=0}^{K+p-1} := \underbrace{\{0, \dots, 0\}}_{p+1}, \underbrace{\{\hat{x}_{k-1}, \hat{x}_k, \dots, 1\}}_{p+1}. \quad (4)$$

More precisely, the i th B-spline basis function, with $1 \leq i \leq K + p - 1$, is constructed using the Cox-de-Boor recursive formulae [5]:

$$B_{i,0}(\xi) := \begin{cases} 1, & \text{if } \gamma_i \leq \xi < \gamma_{i+1}, \\ 0, & \text{otherwise,} \end{cases} \quad (5)$$

$$B_{i,q}(\xi) := \frac{\xi - \gamma_i}{\gamma_{i+q} - \gamma_i} B_{i,q-1}(\xi) + \frac{\gamma_{i+q+1} - \xi}{\gamma_{i+q+1} - \gamma_{i+1}} B_{i+1,q-1}(\xi), \text{ for } 1 \leq q \leq p, \quad (6)$$

where $B_{i,q}(\xi)$ denotes the value of the i th B-spline function of degree q at the point ξ . In formula (6), the limit case $0/0$ is defined as 0. We notice that the Cox-de-Boor recursive formulae (5) and (6) define a total of $K + p$ 1D B-splines basis functions.

We define 3D basis functions by tensor product of 1D B-splines basis functions that, for simplicity, we construct considering the same number of element partitions for all three spatial directions.

For $x = (x_1, x_2, x_3) \in \Omega$, we will denote by

$$B_{\delta,p}(x) = B_{k,p}(x_1) B_{l,p}(x_2) B_{m,p}(x_3), \text{ with } \delta = (\delta_1, \delta_2, \delta_3) \in \mathcal{K}, \quad (7)$$

the evaluation in x of a generic 3D B-spline basis function, where

$$\mathcal{K} = \{0, 1, \dots, K + p - 1\}^3. \quad (8)$$

Remark 1 (Polynomial degree). In the following, we will implicitly consider the polynomial degree p for the spline functions to avoid unnecessary overcharging of notation. That is, we will denote by $B_{\delta}(x) := B_{\delta,p}(x)$, and by $B_l(x_j) := B_{l,p}(x_j)$, for $t = k, l, m$ and $j = 1, 2, 3$, in Eq. (7).

Finally, we will denote by

$$\mathcal{B}_{\mathcal{K}} := \text{span} \{B_{\delta}, \text{ with } \delta \in \mathcal{K}\} \subset H^1(\Omega), \quad (9)$$

the space generated by the 3D-tensor B-spline basis functions of degree p and global regularity $p - 1$.

2.4. Fully-discrete variational formulation

For a given polynomial degree p , the fully-discrete formulation of problem (1) is obtained from (3) by considering the H^1 -conforming space $\mathcal{B}_{\mathcal{K}}$ as the approximation space for the discrete solution $U_{n+1} \approx u_{n+1}$. This is, given $U_n \in \mathcal{B}_{\mathcal{K}}$, we obtain $U_{n+1} \in \mathcal{B}_{\mathcal{K}}$ as the solution of the following discrete variational problem:

$$\text{Find } U_{n+1} \in \mathcal{B}_{\mathcal{K}}, \text{ such that } a(U_{n+1}, B_{\delta}) = l_n(B_{\delta}), \forall \delta \in \mathcal{K}, \quad (10)$$

where

$$a(U_{n+1}, B_{\delta}) = \int_{\Omega} U_{n+1} B_{\delta} \, dx, \quad (11)$$

$$l_n(B_{\delta}) = \int_{\Omega} U_n B_{\delta} \, dx - \Delta_t \int_{\Omega} \nabla U_n \cdot \nabla B_{\delta} \, dx, \quad (12)$$

and U_0 corresponds to the classical L^2 -projection of the initial state u_0 in the B-spline space $\mathcal{B}_{\mathcal{K}}$.

As a consequence of the finite number of basis functions for the discrete space $\mathcal{B}_{\mathcal{K}}$, we can assume that the wanted discrete solution is written as:

$$U_{n+1}(x) = \sum_{\beta \in \{1, \dots, K+p\}^3} \mu_{\beta} B_{\beta}(x). \quad (13)$$

Problem (10) can be equivalently written in matrix form as:

$$\text{Find } \mu \in \mathbb{R}^{(K+p)^3}, \text{ such that } A_{\delta,\beta} \mu = L_{\delta}, \quad (14)$$

with the right-hand side $L_{\delta} = l_n(B_{\delta})$, and the matrix

$$A_{\delta,\beta} = a(B_{\beta}, B_{\delta}). \quad (15)$$

Remark 2 (Linear system representation). For simplicity, we write the matrix A and the right-hand side L of Eq. (15) in terms of multi-indices. However, it must be understood as the resulting system after considering an ordering for the basis functions.

3. Integration algorithms

3.1. Element-by-element integration

For the sake of simplicity, here we will focus on the integration and assembling of the matrix A .

The standard integration strategy consists of assembling the linear system (14) element-by-element. To exemplify the procedure, we assume that the domain Ω is decomposed into a set of K^3 cubic elements.

$$E^{\alpha} = (\hat{x}_i, \hat{x}_{i+1}) \times (\hat{x}_j, \hat{x}_{j+1}) \times (\hat{x}_k, \hat{x}_{k+1}), \quad (16)$$

for element $E \in \Omega$ do
 for test function $B_{i,1}$ do
 for trial function $B_{j,1}$ do
 for test function $B_{i,2}$ do
 for trial function $B_{j,2}$ do
 for test function $B_{i,3}$ do
 for trial function $B_{j,3}$ do
 for quadrature point and weight (ξ, w) in E do
 $A_{\delta,\beta} \leftarrow A_{\delta,\beta} + B_{i,1}(\xi_1) B_{j,1}(\xi_1) B_{i,2}(\xi_2) B_{j,2}(\xi_2) B_{i,3}(\xi_3) B_{j,3}(\xi_3) J(\xi) w;$
Algorithm 1: Classical integration algorithm

where $\hat{x}_j = j/K$ (cf. Section 2.3), and $\alpha = (i, j, k) \in \{1, 2, \dots, K\}^3$.

Denoting by $\beta = (\beta_1, \beta_2, \beta_3)$, and by $\delta = (\delta_1, \delta_2, \delta_3)$, the matrix element $A_{\delta,\beta}$ coming from the discretization of the bilinear form (see (15)) is computed as the sum

$$A_{\delta,\beta} = \sum_{\alpha \in \{1,2,\dots,K\}^3} A_{\delta,\beta}^\alpha, \quad (17)$$

where $A_{\delta,\beta}^\alpha$ is given in terms of the 1D B-spline basis functions as (see (7)):

$$A_{\delta,\beta}^\alpha = \int_{E_\alpha} B_{\beta_1}(x_1) B_{\beta_2}(x_2) B_{\beta_3}(x_3) B_{\delta_1}(x_1) B_{\delta_2}(x_2) B_{\delta_3}(x_3) dx. \quad (18)$$

Let us consider a proper exact quadrature with the particular set of weights and nodes $\{\omega^n = (\omega_1^n, \omega_2^n, \omega_3^n), x^n = (x_1^n, x_2^n, x_3^n) \in E^\alpha\}$, with $n_1 = 1, \dots, P_1$, $n_2 = 1, \dots, P_2$, $n_3 = 1, \dots, P_3$, $n = 1, \dots, P$, and $P = P_1 P_2 P_3$ depending on the quadrature rule and polynomial order p . Then, the matrix element (18) is computed as:

$$A_{\delta,\beta}^\alpha = \sum_{n_1=1}^{P_1} \sum_{n_2=1}^{P_2} \sum_{n_3=1}^{P_3} \omega_1^n \omega_2^n \omega_3^n \Pi(x^n) J(x^n) dx, \quad (19)$$

where $\Pi(x^n) = B_{\beta_1}(x_1^n) B_{\beta_2}(x_2^n) B_{\beta_3}(x_3^n) B_{\delta_1}(x_1^n) B_{\delta_2}(x_2^n) B_{\delta_3}(x_3^n)$ and $J(x^n)$ corresponds to the Jacobian of the particular element evaluated at x^n .

Remark 3 (Element-by-element pre-computations). We notice that (19) can be efficiently calculated by first pre-computing, over each element, only the integral of the B-splines products with non-empty support. Therefore, for a given $\alpha = (i, j, k)$, it will be helpful to introduce the set of multi-indices

$$\mathcal{K}_\alpha = \{(z, r, s) : z \in \{i, \dots, i+p\}, r \in \{j, \dots, j+p\}, s \in \{k, \dots, k+p\}\} \quad (20)$$

corresponding to the indices of the $(p+1)^3$ basis functions with non-empty support in the α -element.

3.2. Algorithm descriptions and computational cost

In this section, we describe the two algorithms to be compared in subsequent sections, the classical integration algorithm, and the sum factorization algorithm.

On one side, in the **classical integration** algorithm, local contributions to the left-hand-side matrix A are represented as a sum over quadrature points, as shown in Eq. (19) and described in Algorithm 1. In this case, the associated computational cost is known that scales, concerning the polynomial degree p , as $\mathcal{O}(p^9)$ [35].

On the other side, **Sum factorization** algorithm consists of reorganizing the integration terms of Eq. (19) to reduce the computational cost, in terms of the polynomial degree p , associated with the sum procedure. In practice, Eq. (19) is written as:

$$A_{\beta,\delta}^\gamma = \sum_{n_3=1}^{P_3} \omega_3^n B_j(x_3^n) B_{m;p}(x_3^n) C(i_2, i_3, j_2, j_3, k_1), \quad (21)$$

where buffer C is given by

$$C(i_2, i_3, j_2, j_3, k_1) = \sum_{n_2=1}^{P_2} \omega_2^n B_i(x_2^n) B_{l;p}(x_2^n) \underbrace{\sum_{n_1=1}^{P_1} \omega_1^n B_h(x_1^n) B_{k;p}(x_1^n) J(x^n)}_{D(i_3, j_3, k_1, k_2)}. \quad (22)$$

The algorithm is described in Algorithm 2. Here we can observe three distinct groups of loops. As a consequence, this implies that the computational cost associated with sum factorization is $\mathcal{O}(p^7)$ [35].

3.3. Concurrency model for sum factorization

Multiple methods are used to verify concurrent computations by creating a concurrency model formally. In [10], a concurrency model based on the *Trace Theory*, introduced by Diekert and Mazurkiewicz [28], is discussed. It contains four levels of concurrency:

1. Concurrent computations on parts of the mesh.
2. Concurrent computations on single elements.
3. Concurrent computations of single entries in an element matrix.
4. Concurrent computations of Cox-de-Boor formulae and 3D B-spline functions evaluation.

Using the same methodology, we will discuss the last two levels of concurrency for the sum factorization algorithm.

The alphabet of tasks for the integration of B-Spline basis functions over a given element consists of the following nine tasks:

1. $t_{\alpha;d}^{0;r,n}$ - computational task evaluating a 1D basis function with subscript r and order 0, over the element E_α at the coordinate of quadrature point x_d^n . Task $t_{\alpha;d}^{0;r,n}$ refers to formula (5). Namely, it computes the function $B_{r,0}(x_d^n)$ over the element E_α .
2. $t_{\alpha;d}^{p;r,n}$, ($p > 0$) - computational task evaluating a 1D basis functions with subscript r and order p , over the element E_α at the coordinate of quadrature point x_d^n . Task $t_{\alpha;d}^{p;r,n}$ refers to formula (6). It contains a series of sums, subtractions, multiplications, and divisions, using output from tasks $t_{\alpha;d}^{p-1;r,n}$ and $t_{\alpha;d}^{p-1;r+1,n}$. Namely, it computes the function $B_r(x_d^n)$ over the element E_α .
3. $s_{\alpha}^{p;n}$ - computational task evaluating the Jacobian value $J(x^n)$ over the element E_α . Namely, it computes $J(x^n)$ according to formula (19).
4. $t_{\alpha;1}^{p;\beta,\gamma;n}$ - computational task evaluating the value of the product of two 1D basis functions B_β^1, B_γ^1 , and the Jacobian value $J(x^n)$, over the element E_α , at the quadrature point x^n . Task $t_{\alpha;1}^{p;\beta,\gamma;n}$ consists of a multiplication of output from tasks $t_{\alpha;1}^{p;\beta;n}$, $t_{\alpha;1}^{p;\gamma;n}$, and $s_{\alpha}^{p;n}$. Namely, it computes buffer $K_{\beta,\gamma;p}(x^n) = B_{\beta;p}(x_1^n) B_{\gamma;p}(x_1^n) J(x^n)$ according to formula (22).
5. $s_{\alpha;1}^{p;\beta,\gamma;n}$ - computational task evaluating the sum of $K_{\beta,\gamma;p}(x^n)$ along x_1 . Task $s_{\alpha;1}^{p;\beta,\gamma;n}$ consists of a sum of outputs from tasks $t_{\alpha;1}^{p;\beta,\gamma;n}$. Namely, it computes the buffer $C_{\beta,\gamma;p}(x^n) = \sum_{n=1}^{P_1} K_{\beta,\gamma;p}(x^n)$ according to formula (22).
6. $t_{\alpha;2}^{p;\beta,\gamma;n}$ - computational task evaluating the value of product of two 1D basis function B_β, B_γ , and sums it with the previous buffer value $C_{\beta,\gamma;p}$, over the element E_α , at the quadrature point x_2^n . Task $t_{\alpha;2}^{p;\beta,\gamma;n}$ consists of a multiplication of output from tasks $t_{\alpha;2}^{p;\beta;n}$, $t_{\alpha;2}^{p;\gamma;n}$, and $t_{\alpha;1}^{p;\beta,\gamma;n}$. Namely, it computes the buffer $F_{\beta,\gamma;p}(x^n) = B_{\alpha,p}(x_2^n) B_{\beta,p}(x_2^n) + C_{\beta,\gamma;p}(x^n)$ according to formula (22).
7. $s_{\alpha;2}^{p;\beta,\gamma;n}$ - computational task evaluating the sum of $F_{\beta,\gamma;p}(x^n)$ along x_1 . Task $s_{\alpha;2}^{p;\beta,\gamma;n}$ consists of a sum of outputs from tasks

```

for test function  $B_{i,3} - (i_3)$  do
  for trial function  $B_{j,3} - (j_3)$  do
    for quadrature point  $(\xi_1, w_1)$  in  $E - (k_1)$  do
      for quadrature point  $(\xi_2, w_2)$  in  $E - (k_2)$  do
        for quadrature point  $(\xi_3, w_3)$  in  $E - (k_3)$  do
           $D(i_3, j_3, k_1, k_2) \leftarrow D(i_3, j_3, k_1, k_2) + B_{i,3}(\xi_3) B_{j,3}(\xi_3) w_3 J(\xi);$ 
for test function  $B_{i,2} - (i_2)$  do
  for trial function  $B_{j,2} - (j_2)$  do
    for test function  $B_{i,3} - (i_3)$  do
      for trial function  $B_{j,3} - (j_3)$  do
        for quadrature point  $(\xi_1, w_1)$  in  $E - (k_1)$  do
          for quadrature point  $(\xi_2, w_2)$  in  $E - (k_2)$  do
             $C(i_2, i_3, j_2, j_3, k_1) \leftarrow C(i_2, i_3, j_2, j_3, k_1) + B_{i,2}(\xi_2) B_{j,2}(\xi_2) D(i_3, j_3, k_1, k_2) w_2;$ 
for test function  $B_{i,1} - (i_1)$  do
  for trial function  $B_{j,1} - (j_1)$  do
    for test function  $B_{i,2} - (i_2)$  do
      for trial function  $B_{j,2} - (j_2)$  do
        for test function  $B_{i,3} - (i_3)$  do
          for trial function  $B_{j,3} - (j_3)$  do
            for quadrature point  $(\xi_1, w_1)$  in  $E - (k_1)$  do
               $A(i_1, j_1) \leftarrow A(i_1, j_1) + B_{i,1}(\xi_1) B_{j,1}(\xi_1) C(i_2, i_3, j_2, j_3, k_1) w_1;$ 

```

Algorithm 2: Sum factorization algorithm

- $t_{\alpha;2}^{p;\beta;\gamma;n}$. Namely, it computes the buffer $D_{\beta,\gamma;p}(x^n) = \sum_{n=1}^{P_2} F_{\beta,\gamma;p}(x^n)$ according to formula (22).
8. $t_{\alpha;3}^{p;\beta;\gamma;n}$ - computational task evaluating the value of the product of two 1D basis functions B_β , B_γ , and sums it with the previous buffer value $D_{\beta,\gamma;p}$, over element E_α , at the quadrature point x_3^n . Task $t_{\alpha;3}^{p;\beta;\gamma;n}$ consists of a multiplication of outputs from tasks $t_{\alpha;2}^{p;\beta;\gamma;n}$, $t_{\alpha;3}^{p;\beta;\gamma;n}$, and $t_{\alpha;2}^{p;\beta;\gamma;n}$. Namely, it computes $H_{\beta,\gamma}^\alpha = B_{\alpha;p}(x_3^n) B_{\beta;p}(x_3^n) + D_{\beta,\gamma;p}(x^n)$ according to formula (21).
9. $s_{\alpha;3}^{p;\beta;\gamma;n}$ - computational task evaluating the sum of $E_{\beta,\gamma;p}(x^n)$ along x_1 . Task $s_{\alpha;3}^{p;\beta;\gamma;n}$ consists of a sum of outputs from tasks $t_{\alpha;1}^{p;\beta;\gamma;n}$. Namely, it computes the buffer $A_{\beta,\gamma} = \sum_{n=1}^{P_3} H_{\beta,\gamma;p}(x^n)$ according to formula (21).

Summarizing, each task has two, three, or four upper subscripts in two or three groups divided by a semicolon. The first group p determines the B-spline order. The second group (optional) of multi indices β , γ or index r determines B-spline functions indices. The third (optional) group n determines quadrature point x^n , at which the functions are evaluated. Additionally, tasks have one or two bottom subscripts. The first one, with the index α , determines the element over which we perform computations. The second (optional) determines the direction in the x , y , or z axis. It is important to recall that a particular task cannot be performed until the completion of the tasks for which its output is required.

3.3.1. Set of dependencies

In this section, we define the alphabet of tasks Σ and the set of dependencies between them denoted by D . For this, we start by setting the variables:

$$\begin{aligned}
 n &\in \{1, 2, \dots, P\}, \\
 q &\in \{0, 1, \dots, p\}, \\
 d &\in \{1, 2, 3\}, \\
 f &\in \{k, k+1, \dots, k+p\}, \\
 g &\in \{l, l+1, \dots, l+p\}, \\
 h &\in \{m, m+1, \dots, m+p\}, \\
 \alpha &= (k, l, m) \in \mathcal{K}, \\
 \beta &= (a, b, c) \in \mathcal{K}^\alpha, \\
 \gamma &\in \mathcal{K}^\alpha.
 \end{aligned} \tag{23}$$

We also set I_α as the function computing the index in the local element (E_α) matrix based on the multi index (a, b, c) . This is,

$$I_\alpha : \mathcal{K}^\alpha \rightarrow \{0, 1, 2, \dots, (p+1)^3\}. \tag{24}$$

We define the alphabet of tasks as:

$$\Sigma = \left\{ t_{\alpha;1}^{q;f;n}, t_{\alpha;2}^{q;g;n}, t_{\alpha;3}^{q;h;n}, t_{\alpha}^{p;\beta;\gamma;n}, s_{\alpha}^{p;\beta;\gamma;n} \right\} \cup \left\{ t_{\alpha;d}^{p;\beta;\gamma;n}, s_{\alpha;d}^{p;\beta;\gamma;n}; I_\alpha(\beta) \geq I_\alpha(\gamma) \right\}, \tag{25}$$

and the set of dependencies between tasks from the alphabet Σ as:

$$D = J^+ \cup (J^+)^{-1} \cup I_\Sigma, \tag{26}$$

where

$$J = J_1 \cup J_2 \cup J_3 \cup J_4, \tag{27}$$

with

$$\begin{aligned}
 J_1 &= \left\{ (t_{\alpha;1}^{q-1;f;n}, t_{\alpha;1}^{q;f;n}), (t_{\alpha;1}^{q-1;f+1;n}, t_{\alpha;1}^{q;f;n}), (t_{\alpha;2}^{q-1;g;n}, t_{\alpha;2}^{q;g;n}), \right. \\
 &\quad \left. (t_{\alpha;2}^{q-1;g+1;n}, t_{\alpha;2}^{q;g;n}), (t_{\alpha;3}^{q-1;h;n}, t_{\alpha;3}^{q;h;n}), (t_{\alpha;3}^{q-1;h+1;n}, t_{\alpha;3}^{q;h;n}) \right\}, \\
 J_2 &= \left\{ (t_{\alpha;1}^{p;\alpha;n}, t_{\alpha;1}^{p;\beta;\gamma;n}), (t_{\alpha;2}^{p;\alpha;n}, t_{\alpha;2}^{p;\beta;\gamma;n}), (t_{\alpha;3}^{p;\alpha;n}, t_{\alpha;3}^{p;\beta;\gamma;n}) \right\}, \\
 J_3 &= \left\{ (t_{\alpha;1}^{p;\beta;\gamma;n}, s_{\alpha;1}^{p;\beta;\gamma;n}), (t_{\alpha;2}^{p;\beta;\gamma;n}, s_{\alpha;2}^{p;\beta;\gamma;n}), (t_{\alpha;3}^{p;\beta;\gamma;n}, s_{\alpha;3}^{p;\beta;\gamma;n}) \right\}, \\
 J_4 &= \left\{ (s_{\alpha}^{p;\beta;\gamma;n}, t_{\alpha;1}^{p;\beta;\gamma;n}), (s_{\alpha}^{p;\beta;\gamma;n}, t_{\alpha;2}^{p;\beta;\gamma;n}), (s_{\alpha}^{p;\beta;\gamma;n}, t_{\alpha;3}^{p;\beta;\gamma;n}) \right\},
 \end{aligned}$$

and $(\cdot)^+$, $(\cdot)^{-1}$, I_Σ represent the transitive closure, the inverse and the identity relation, respectively.

Primitives described above define the monoid of traces for the problems under consideration. J defined in Eq. (27) will stand for edges in Diekert dependency graph [28], which will be drawn later in frame of this model in Figs. 2 and 8.

After building the primitives of the trace monoid, i.e., the alphabet of tasks (25) and the dependency relation (26), we define the pseudocode allowing to compute the value of integral (18), presented in Tables 1–3, that we have split into three parts to facilitate its reading. The dependencies in this algorithm's record determine only the sequence of operations in one string representing the desired trace. The alphabet of tasks Σ (25), the dependencies relation D (26), and the trace defined by pseudocode (Tables 1–3) allow us to compute the Diekert dependency graph, which is convenient for the correct and effective scheduling of tasks in a heterogeneous computer environment.

Table 1The algorithm generating sample string of tasks representing sum factorization for the matrix A . Part 1.

```

1 BEGIN
2 //loop over elements
3 FOREACH  $\{\alpha := (k, l, m)\} \in \mathcal{K}$ 
4   //compute local element matrix
5   element_matrix = zeros( $(p+1)^3, (p+1)^3$ )
6   local_matrix = zeros( $(p+1)^3, (p+1)^3, P$ )
7   local_C_matrix = zeros( $P_2, P_3, p+1, p+1, P_1$ )
8   element_C_matrix = zeros( $P_2, P_3, p+1, p+1$ )
9   local_D_matrix = zeros( $P_3, p+1, p+1, p+1, p+1, P_1$ )
10  element_D_matrix = zeros( $P_3, p+1, p+1, p+1, p+1$ )
11
12  //loop over quadrature points
13  FOR  $n_1=1, P_1$ 
14    1D_matrix = zeros( $p+1$ )
15    //compute 1D functions
16    FOR  $r=0, p$ 
17       $t_{a;1}^{p;k,n}$ : 1D_matrix( $r$ ) = compute recursive ( $B_{k+r,p}(x_1^n)$ )
18    ENDFOR
19    FOR  $n_2=1, P_2$ 
20      FOR  $n_3=1, P_3$ 
21         $s_a^{p;n}$ :  $c = J(x^n)$ 
22        //compute product of two functions
23        FOREACH  $\beta = (a, :, :) \in \mathcal{K}^\alpha$ 
24           $i = \text{index\_in\_local\_matrix}(a, :, :)$ 
25          FOREACH  $\gamma = (d, :, :) \in \mathcal{K}^\alpha$ 
26             $j = \text{index\_in\_local\_matrix}(d, :, :)$ 
27             $t_{a;1}^{p;\beta,\gamma,n}$ : local_C_matrix( $n_2, n_3, i, j, n_1$ ) =
28              = 1D_matrix( $a$ ) * 1D_matrix( $d$ ) *  $c$ 
29          ENDFOR
30        ENDFOR
31      ENDFOR
32    ENDFOR
33  ENDFOR
34  //sum local components from each quadrature point
35  FOREACH  $\beta = (a, :, :) \in \mathcal{K}^\alpha$ 
36     $i = \text{index\_in\_local\_matrix}(a, :, :)$ 
37    FOREACH  $\gamma = (d, :, :) \in \mathcal{K}^\alpha$ 
38       $j = \text{index\_in\_local\_matrix}(d, :, :)$ 
39       $s_{a;1}^{p;\beta,\gamma}$ : element_C_matrix( $n_2, n_3, i, j$ ) =
40        = reduction(local_C_matrix( $n_2, n_3, i, j, :$ ), +)
41    ENDFOR
42  ENDFOR

```

3.4. Application of trace theory to sum factorization

This section describes the methodology for creating the Diekert Dependency Graph (DDG) and the Foata Normal Form (FNF), applied to the sum factorization integration method of p -order B-spline basis functions. (DDG) presents all computational tasks performed in computation and dependencies between them. Within (DDG) and FNF, we can distinguish Foata classes, which help with practically implementing concurrent computations.

For a given polynomial degree p , there are $(p+1)^3$ basis functions with non-empty support over each cubic element E_α , with $\alpha \in \mathcal{K}$. Therefore, for every E_α , we require to construct a element matrix of size $(p+1)^3 \times (p+1)^3$, according to Eq. (18). However, it matrix A is symmetric, it is not necessary to compute the full element matrix. Indeed, we only require to compute $(p+1 + (p+1)^3 \times (p+1)^3)/2$ matrix entries.

To exemplify the cost associated with the computation of a single entry in the matrix A , let us assume that a quadrature of $P = P_1 P_2 P_3$ points per element is employed. Let us also denote by $x^1, x^2, \dots, x^P$ the corresponding quadrature points, where $x^n = (x_1^n, x_2^n, x_3^n)$, for $n = 1, \dots, P$.

In the procedure for each quadrature point, we start by computing $(p+1)$ 1D functions in each direction ($3p+3$ functions in total) employing the Cox-de-Boor formulae (Classes 0, 1, $\dots, p$ in Fig. 2). This completes

all tasks of type $t_{a,d}^{0;r,n}, t_{a,d}^{1;r,n}$ up to $t_{a,d}^{p;r,n}$ (see Table 4). Within the class p , we include one extra task computing $s_a^{p;n}$.

The class $p+1$, in Fig. 4, completes all tasks of the type $t_{a;1}^{p;\beta,\gamma,n}$ (see Table 4).

The concurrently computed components can be summed to evaluate scalar products of the 1D basis functions over the element E_α , $(k, l, m) = \alpha \in \mathcal{K}$, which completes all tasks of the type $s_{a;1}^{p;\beta,\gamma}$ (see Table 4).

Next, we construct two pairs of classes $p+3$ and $p+4$ (Figs. 5, 6), and $p+5$ and $p+6$ (Figs. 7, 8), in similar manner to classes $p+1$ and $p+2$. Finally, we present all tasks in Tables 4 and 5.

3.5. Scheduling algorithm

To obtain a similar scheduling quality to the classical algorithm on massively parallel shared-memory machines, we employ the Foata-Normal-Form (FNF) [36]. The Diekert dependency graphs (see Section 3.4) show the consecutive Foata classes for each considered case of sum factorization. Within a given Foata class, tasks can be executed in any order. Completion of the entire previous Foata class is a sufficient condition to begin the computation of the next one. The proposed strategy ensures no deadlocks, high-quality scheduling, and no need for intra-class synchronization.

Based on Figs. 2–4, we can describe a general procedure for creating subsequent Foata classes, containing the following tasks:

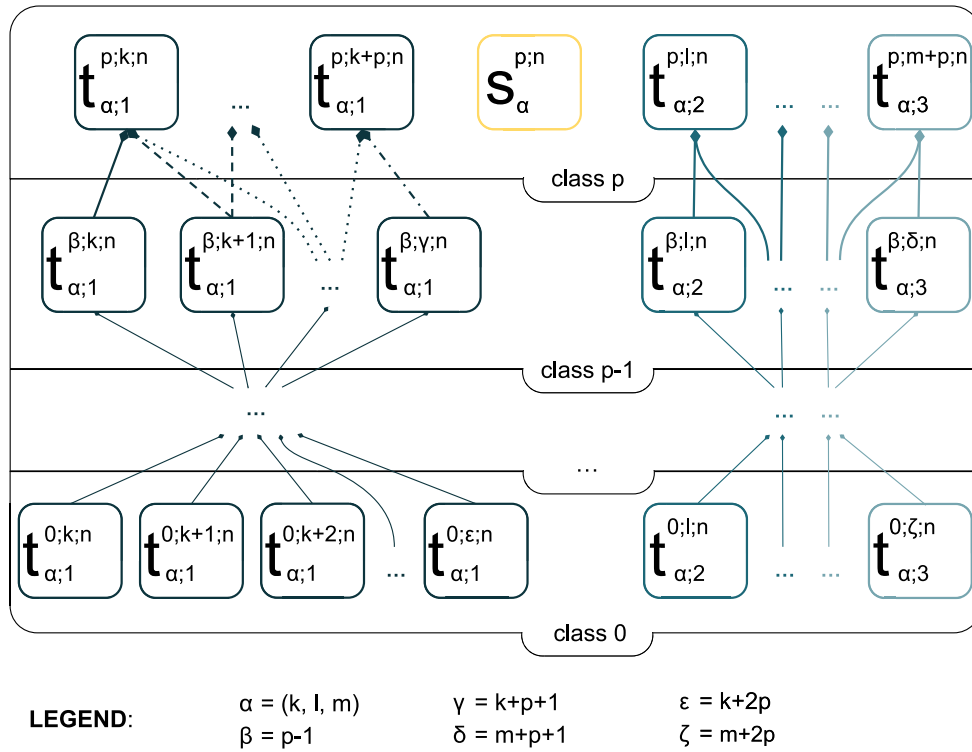


Fig. 2. Relationships between classes 0 to p for p -order functions. Tasks belonging to one class correspond to going through one iteration of the Cox-de Boor recursion formulae ((5), (6)) for each of the three dimensions of the model. The dimensions are differentiated by color.

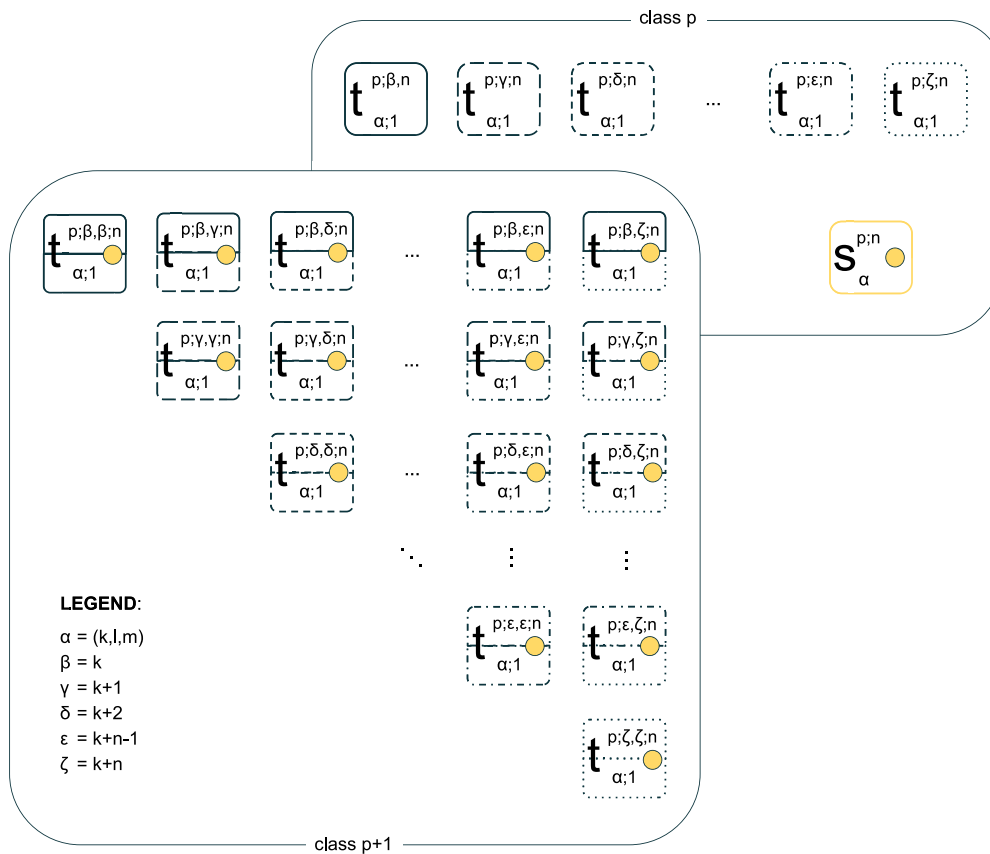


Fig. 3. Relationships between classes p and $p+1$ for p -order functions. Each task in class $p+1$ corresponds to the dot product of two 1D B-spline functions, so it depends on the two tasks in class p . To maintain the transparency of the chart, the relationships between the second and third classes are marked with a border type.

Table 2

The algorithm generating sample string of tasks representing sum factorization for the matrix A . Part 2.

```

1 //loop over quadrature points
2 FOR  $n_2=1, P_2$ 
3   1D_matrix = zeros( $p+1$ )
4   //compute 1D functions
5   FOR  $r=0, p$ 
6      $t_{\alpha,2}^{p;k,n}$ : 1D_matrix( $r$ ) = comformulaJ3pute recursive ( $B_{k+r,p}(x_2^n)$ )
7   ENDFOR
8   FOR  $n_3=1, P_3$ 
9     //compute product of two functions
10    FOREACH  $\beta = (a, b, :) \in \mathcal{K}^a$ 
11       $[i_1, i_2] = \text{index\_in\_local\_matrix}(a, b, :)$ 
12      FOREACH  $\gamma = (d, e, :) \in \mathcal{K}^a$ 
13         $[j_1, j_2] = \text{index\_in\_local\_matrix}(d, e, :)$ 
14         $t_{\alpha,2}^{p;\beta,\gamma,n}$ : local_D_matrix( $n_3, i_1, i_2, j_1, j_2, n_2$ ) =
15          = 1D_matrix( $b$ ) * 1D_matrix( $e$ ) *
16            * element_C_matrix( $n_2, n_3, i_1, j_1$ )
17      ENDFOR
18    ENDFOR
19  ENDFOR
20 ENDFOR
21 //sum local components from each quadrature point
22 FOREACH  $\beta = (a, b, :) \in \mathcal{K}^a$ 
23    $[i_1, i_2] = \text{index\_in\_local\_matrix}(a, b, :)$ 
24   FOREACH  $\gamma = (d, e, :) \in \mathcal{K}^a$ 
25      $[j_1, j_2] = \text{index\_in\_local\_matrix}(d, e, :)$ 
26      $s_{\alpha,2}^{p;\beta,\gamma}$ : element_D_matrix( $n_3, i_1, i_2, j_1, j_2$ ) =
27       = reduction(local_D_matrix( $n_3, i_1, i_2, j_1, j_2, :$ ), +)
28   ENDFOR
29 ENDFOR

```

Table 3

The algorithm generating sample string of tasks representing sum factorization for the matrix A . Part 3.

```

1 //loop over quadrature points
2 FOR  $n_3=1, P_3$ 
3   1D_matrix = zeros( $p+1$ )
4   //compute 1D functions
5   FOR  $r=0, p$ 
6      $t_{\alpha,3}^{p;k,n}$ : 1D_matrix( $r$ ) = compute recursive ( $B_{k+r,p}(x_3^n)$ )
7   ENDFOR
8   //compute product of two functions
9   FOREACH  $\beta = (a, b, c) \in \mathcal{K}^a$ 
10     $[i; i_1, i_2, i_3] = \text{index\_in\_local\_matrix}(a, b, c)$ 
11    FOREACH  $\gamma = (d, e, f) \in \mathcal{K}^a$ 
12       $[j; j_1, j_2, j_3] = \text{index\_in\_local\_matrix}(d, e, f)$ 
13       $t_{\alpha,3}^{p;\beta,\gamma,n}$ : local_matrix( $i, j, n_3$ ) =
14        = 1D_matrix( $c$ ) * 1D_matrix( $f$ ) *
15          * element_D_matrix( $n_3, i_1, i_2, j_1, j_2$ )
16    ENDFOR
17  ENDFOR
18 ENDFOR
19 //sum local components from each quadrature point
20 FOREACH  $\beta = (a, b, c) \in \mathcal{K}^a$ 
21    $[i; i_1, i_2, i_3] = \text{index\_in\_local\_matrix}(a, b, c)$ 
22   FOREACH  $\gamma = (d, e, f) \in \mathcal{K}^a$ 
23      $[j; j_1, j_2, j_3] = \text{index\_in\_local\_matrix}(d, e, f)$ 
24      $s_{\alpha,3}^{p;\beta,\gamma}$ : element_matrix( $i, j$ ) =
25       = reduction(local_matrix( $i, j, n_3$ ), +)
26   ENDFOR
27 ENDFOR
28
29
30 //insert local matrices into global ones
31 insert_local_element_2_global(element_matrix,  $\alpha$ )
32 ENDFOR
33 END

```

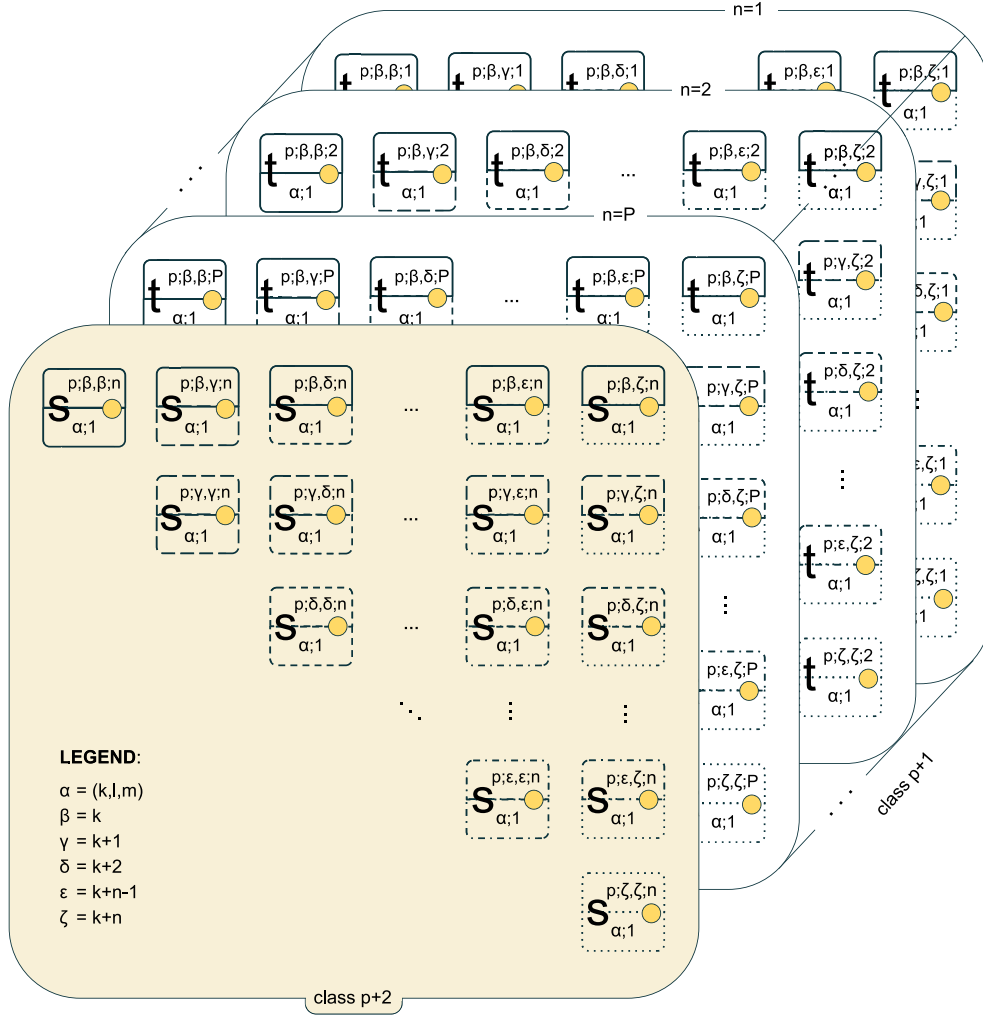


Fig. 4. Relationships between classes $p+1$ and $p+2$ for p -order functions. Each task from class $p+3$ corresponds to the approximation of the function value using Gaussian quadrature, therefore it depends on P tasks from class $p+1$. The task s depends on all tasks t with regards to its distribution in subsequent sheets $1, 2, \dots, P$.

Table 4

Computational tasks for performing computations of sum factorization algorithm of 3D order p basis functions over element E_α , $(k, l, m) = \alpha \in \mathcal{K}$. Part 1.

$s_{\alpha;3}^{p;\beta}$	$B_\beta(x^n) = B_{m;p}(x_3^n) B_{c;p}(x_3^n)$	$n \in \{1, 2, \dots, P\},$ $\beta = (k, l, m) \in \mathcal{K}^\alpha$ $\gamma = (a, b, c) \in \mathcal{K}^\alpha$ $I(\beta) \geq I(\gamma)$
$t_{\alpha;3}^{p;\beta;n}$	$B_\beta(x^n) = B_{m;p}(x_3^n) B_{c;p}(x_3^n)$	$n \in \{1, 2, \dots, P\},$ $\beta = (k, l, m) \in \mathcal{K}^\alpha$ $\gamma = (a, b, c) \in \mathcal{K}^\alpha$ $I(\beta) \geq I(\gamma)$
$s_{\alpha;2}^{p;\beta}$	$B_\beta(x^n) = B_{l;p}(x_2^n) B_{b;p}(x_2^n)$	$n \in \{1, 2, \dots, P\},$ $\beta = (k, l, m) \in \mathcal{K}^\alpha$ $\gamma = (a, b, c) \in \mathcal{K}^\alpha$ $I(\beta) \geq I(\gamma)$
$t_{\alpha;2}^{p;\beta;n}$	$B_\beta(x^n) = B_{l;p}(x_2^n) B_{b;p}(x_2^n)$	$n \in \{1, 2, \dots, P\},$ $\beta = (k, l, m) \in \mathcal{K}^\alpha$ $\gamma = (a, b, c) \in \mathcal{K}^\alpha$ $I(\beta) \geq I(\gamma)$

Table 5

Computational tasks for performing computations of sum factorization algorithm of 3D order p basis functions over element E_α , $(k, l, m) = \alpha \in \mathcal{K}$. Part2.

$s_{\alpha;1}^{p;\beta}$	$B_\beta(x^n) = B_{k;p}(x_1^n) B_{a;p}(x_1^n)$	$n \in \{1, 2, \dots, P\},$ $\beta = (k, l, m) \in \mathcal{K}^\alpha$ $\gamma = (a, b, c) \in \mathcal{K}^\alpha$ $I(\beta) \geq I(\gamma)$
$t_{\alpha;1}^{p;\beta;n}$	$B_\beta(x^n) = B_{k;p}(x_1^n) B_{a;p}(x_1^n)$	$n \in \{1, 2, \dots, P\},$ $\beta = (k, l, m) \in \mathcal{K}^\alpha$ $\gamma = (a, b, c) \in \mathcal{K}^\alpha$ $I(\beta) \geq I(\gamma)$
$t_{\alpha;d}^{p;r;n}$	$B_r(x_d^n)$	$d \in \{1, 2, 3\},$ $n \in \{1, 2, \dots, P\}$
$t_{\alpha;d}^{1;r;n}$	$B_{r,1}(x_d^n)$	$d \in \{1, 2, 3\},$ $n \in \{1, 2, \dots, P\}$
$t_{\alpha;d}^{0;r;n}$	$B_{r,0}(x_d^n)$	$d \in \{1, 2, 3\},$ $n \in \{1, 2, \dots, P\}$

- Class m , where $m \in \{0, \dots, p-1\}$

$$\{t_{\alpha;d}^{m;r;n}; d \in \{1, 2, 3\}, n \in \{1, 2, \dots, P\}\}, \quad (28)$$

- Class p

$$\{t_{\alpha;d}^{p;r;n}, s_{\alpha}^{p,n}; d \in \{1, 2, 3\}, n \in \{1, 2, \dots, P\}\}, \quad (29)$$

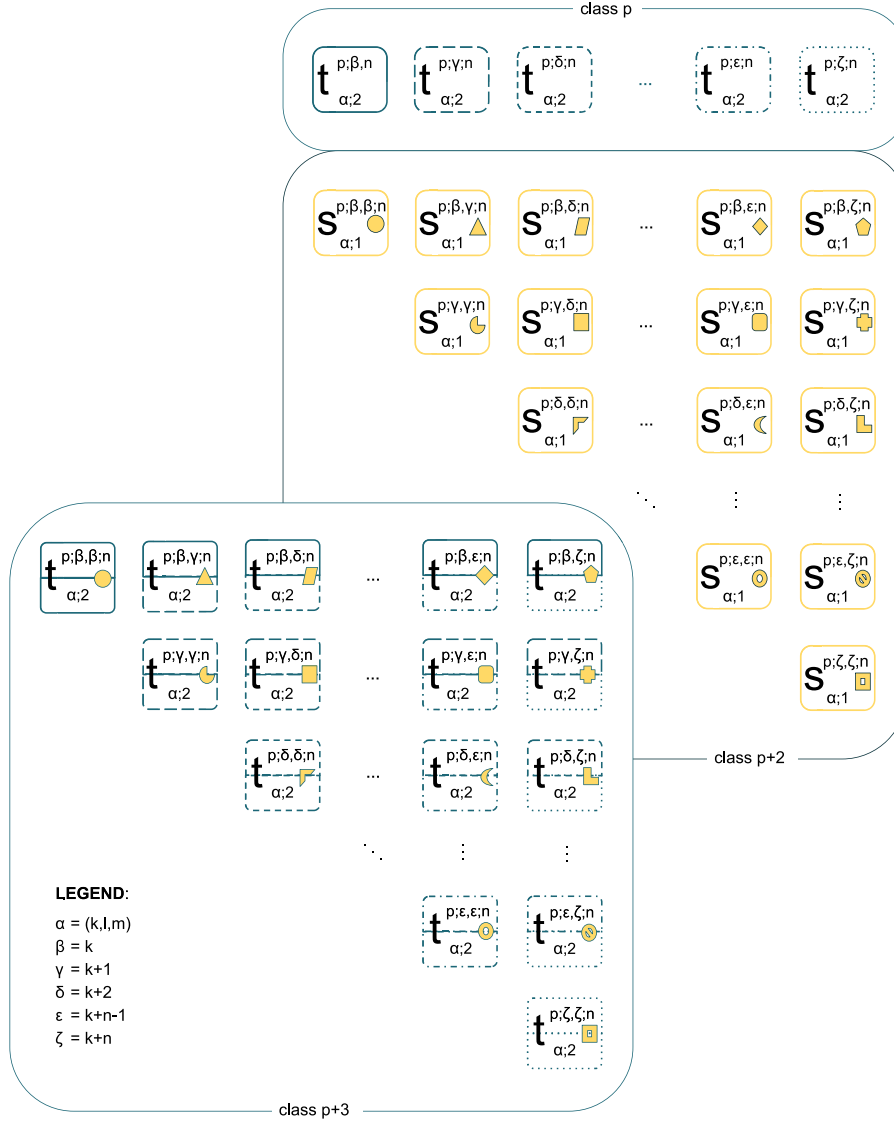


Fig. 5. Relationships between classes p , $p+2$, and $p+3$ for p -order functions. Each task in class $p+1$ corresponds to the dot product of two 1D B-spline functions and a sum, so it depends on the two tasks in class p and some tasks from class $p+2$. To maintain the transparency of the chart, the relationships between the second and third classes are marked with a border type.

• Class $p+1$

$$\{t_{\alpha;1}^{p;\beta;\gamma;n}; (\beta, \gamma) \in \mathcal{K}^\alpha \times \mathcal{K}^\alpha, I(\beta) \geq I(\gamma), n \in \{1, 2, \dots, P\}\}, \quad (30)$$

• Class $p+2$

$$\{s_{\alpha;1}^{p;\beta;\gamma}; (\beta, \gamma) \in \mathcal{K}^\alpha \times \mathcal{K}^\alpha, I(\beta) \geq I(\gamma), n \in \{1, 2, \dots, P\}\}, \quad (31)$$

• Class $p+3$

$$\{t_{\alpha;2}^{p;\beta;\gamma;n}; (\beta, \gamma) \in \mathcal{K}^\alpha \times \mathcal{K}^\alpha, I(\beta) \geq I(\gamma), n \in \{1, 2, \dots, P\}\}, \quad (32)$$

• Class $p+4$

$$\{s_{\alpha;2}^{p;\beta;\gamma}; (\beta, \gamma) \in \mathcal{K}^\alpha \times \mathcal{K}^\alpha, I(\beta) \geq I(\gamma), n \in \{1, 2, \dots, P\}\}, \quad (33)$$

• Class $p+5$

$$\{t_{\alpha;3}^{p;\beta;\gamma;n}; (\beta, \gamma) \in \mathcal{K}^\alpha \times \mathcal{K}^\alpha, I(\beta) \geq I(\gamma), n \in \{1, 2, \dots, P\}\}, \quad (34)$$

• Class $p+6$

$$\{s_{\alpha;3}^{p;\beta;\gamma}; (\beta, \gamma) \in \mathcal{K}^\alpha \times \mathcal{K}^\alpha, I(\beta) \geq I(\gamma), n \in \{1, 2, \dots, P\}\}, \quad (35)$$

The first Foata classes ((28), (30)) are responsible for valuating the values of 1D n -order basis functions over the element E_α , at the Gaussian quadrature points, using recursive Cox-de-Boor formulae ((5), (6)) and the Jacobian (30). Subsequent Foata classes of two kinds follow this:

1. Computational tasks ((30), (32), (34)) evaluate the dot products of 1D n -order basis functions over the element E_α at Gaussian quadrature point,
2. computational tasks ((31), (33), (35)) evaluate the dot products of 1D n -order basis functions over the element E_α and buffers.

All the tasks mentioned above are performed on a homogeneous architecture. Thus, we can expect near-identical execution time for each of them inside a particular Foata class. Consequently, all tasks from the particular Foata class can be effectively scheduled as a common bag.

Over each element E_α we repeat the same procedure of invoking tasks using parameters associated with this element. We invoke Foata classes starting from the Foata class 0, and each time wait for all tasks to be completed before invoking the next Foata class. Using a

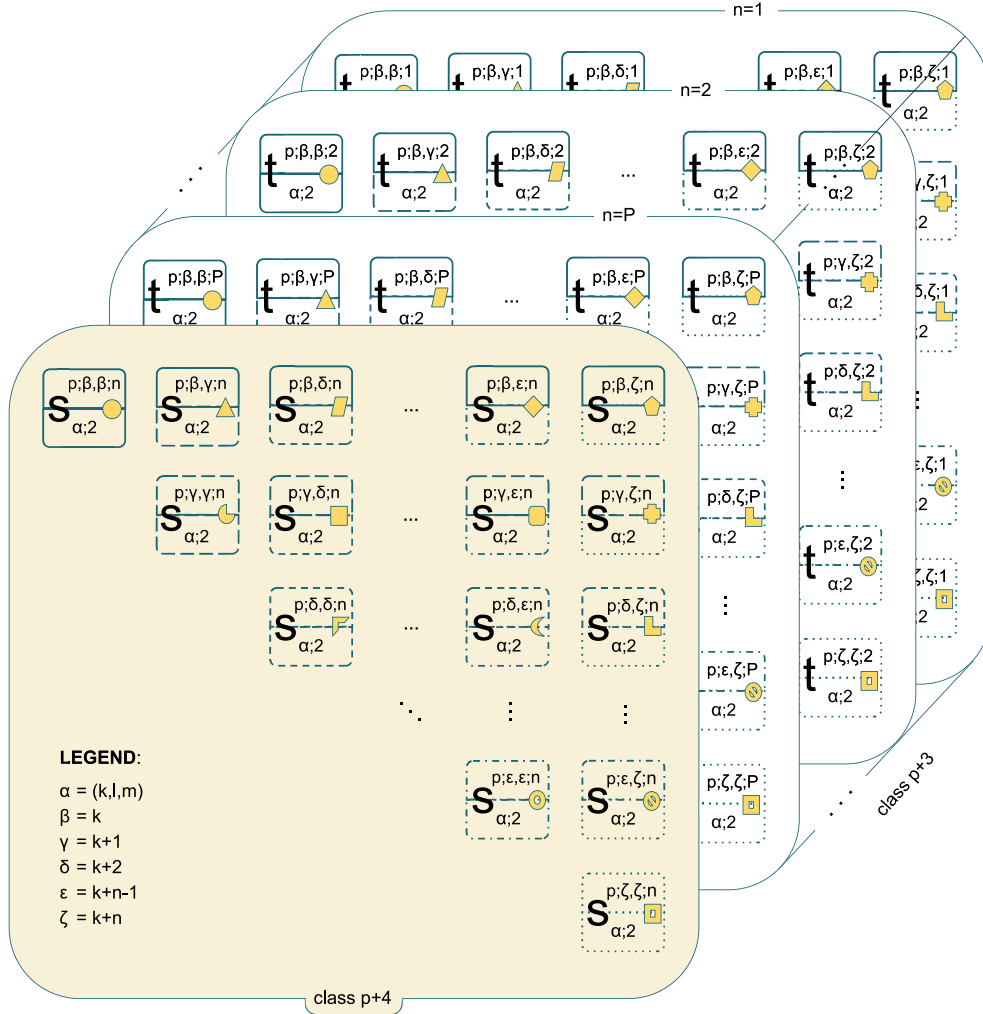


Fig. 6. Relationships between classes $p+3$ and $p+4$ for p -order functions. Each task from class $p+3$ corresponds to the approximation of the function value using Gaussian quadrature, therefore it depends on P tasks from class $p+3$. The task s depends on all tasks t with regards to its distribution in subsequent sheets $1, 2, \dots, P$.

simplified scheduling method, based on FNF and the proposed above, despite having no theoretical proof, results in near-optimal performance in practical applications while maintaining a relatively simple implementation.

4. Numerical results

Now, we compare the computational performance of parallel integration using the classical algorithm and sum factorization. In both cases, implementation was done in Fortran 2003, using OpenMP for loop parallelization. The measurements concern the execution time for the sequential integration algorithm executed on CPU and the concurrent integration algorithm run on a shared memory CPU with 12 cores. Computations were performed on a Banach Linux workstation equipped with AMD Ryzen 9 3900X processor and 64 GB RAM. It is worth noting that the CPU, despite having 3.8 GHz base clock speed and 4.6 GHz boost, was working at a constant 4.0 GHz in the multi-threaded (12 cores) workload and at 4.1 GHz in single-threaded workload (1 core). The computations have been performed using the code compiled with ifort with -O2 level of optimization.

In Sections 4.1, 4.2, and 4.3 we present the experimental results. In Section 4.4 we discuss obtained results.

4.1. Inside element scalability

We first performed computations with parallelization inside an element, then sequential looping over elements. In such a case, we consider a mesh of 30^3 elements. The comparison of the speedup for different polynomial orders is presented in Figs. 9 and 10. Finally, in Figs. 11 and 12 we presented efficiency for the classical integration algorithm and sum factorization respectively.

4.2. Over element scalability

As a second experiment, we performed computations with sequential computations inside the element and parallel looping over elements. In this case, we also used a mesh of 30^3 elements.

The comparison of speedup for different polynomial orders is presented in Figs. 13 and 14. Finally, in Figs. 15 and 16 we present the efficiency for the classical integration algorithm and sum factorization respectively.

4.3. Speedup limits

As a final experiment, we estimate the maximum speedup for both the parallelization schemes (see Sections 4.1 and 4.2), and also its

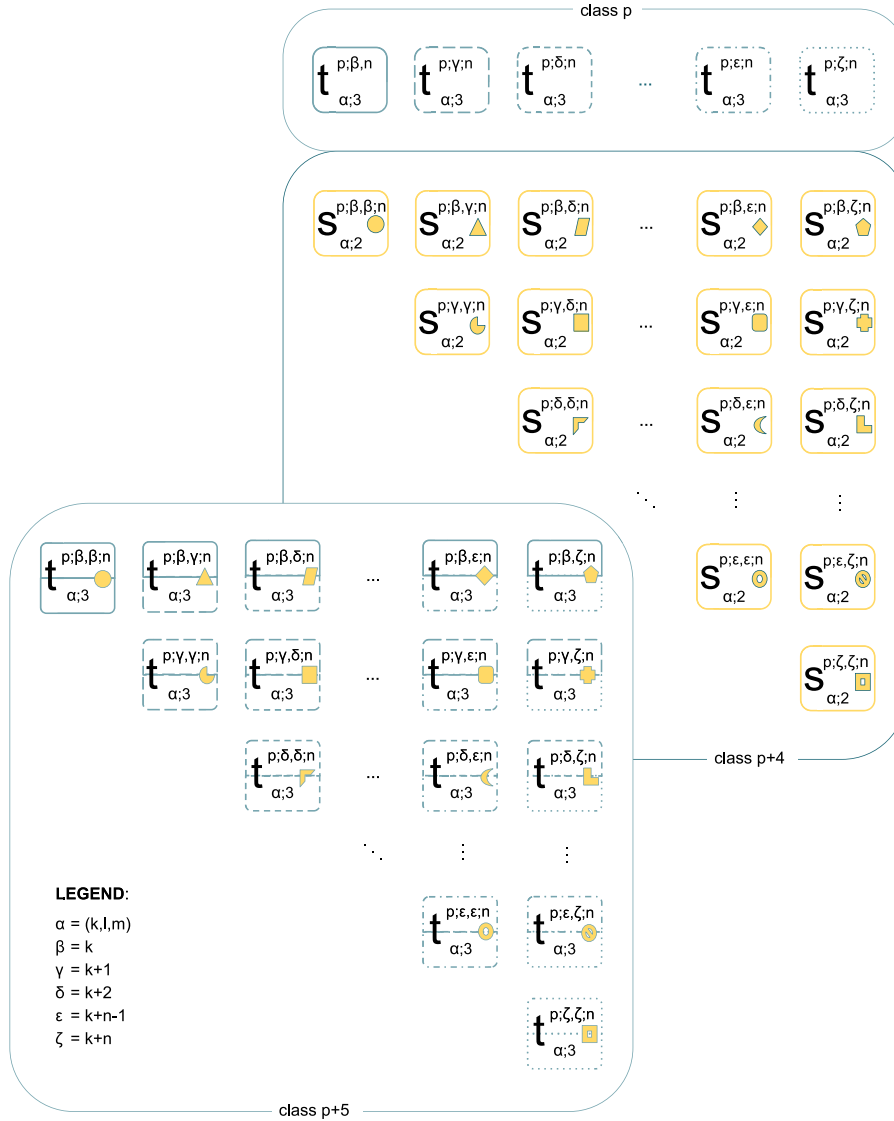


Fig. 7. Relationships between classes p , $p+4$, and $p+5$ for p -order functions. Each task in class $p+5$ corresponds to the dot product of two 1D B-spline functions and a sum, so it depends on the two tasks in class p and some tasks from class $p+4$. To maintain the transparency of the chart, the relationships between the second and third classes are marked with a border type.

combination. When considering integration inside a single element, the problem size is fixed regardless of the mesh size. Amdahl's law is appropriate for this kind of scenario. Therefore, to find the percentage of the algorithm which benefits from speedup $\mathcal{P}$, we invoke the Amdahl's equation:

$$S(\nu) = \frac{1}{(1 - \mathcal{P}) + \frac{\mathcal{P}}{\nu}} \quad (36)$$

where $\mathcal{P}$ denotes the percentage of the algorithm which benefits from the parallel speedup, ν is the number of threads, and $S(\nu)$ is the measured speedup when using ν threads.

From the previous equation, we can derive the value of $\mathcal{P}$ and the speedup limit, which are explicitly given by:

$$\mathcal{P} = \frac{\frac{\nu}{S(\nu)} - \nu}{1 - \nu}, \quad (37)$$

and

$$S(\infty) = \lim_{\nu \rightarrow \infty} \frac{1}{(1 - \mathcal{P}) + \frac{\mathcal{P}}{\nu}} = \frac{1}{1 - \mathcal{P}}, \quad (38)$$

respectively. For different values of p , we consider the maximum experimental speedup observed from the numerical results for both methods. Next, using Eqs. (37) and (38), we computed the percentage of algorithm that benefits from the parallel speedup and the theoretical maximum speedup. Finally, we estimated the combined maximum speedup by assuming two layers of parallelism. This is, one layer representing the scheme of Section 4.1, and another representing the scheme of Section 4.2. The results for the classical integration algorithm are presented in Table 6, while the results for sum factorization in Table 7, where t_{base} corresponds to the measured time for the computations on a single core.

4.4. Discussion of the numerical results

For different values of p , we consider the maximum experimental speedup observed from the numerical results for both methods. Next, using Eqs. (37) and (38), we computed the percentage of algorithm that benefits from the parallel speedup and the theoretical maximum speedup. The results are presented in Tables 6, and 7 for classical and sumfactorization algorithm, respectively. From Figs. 9 and 13 we can

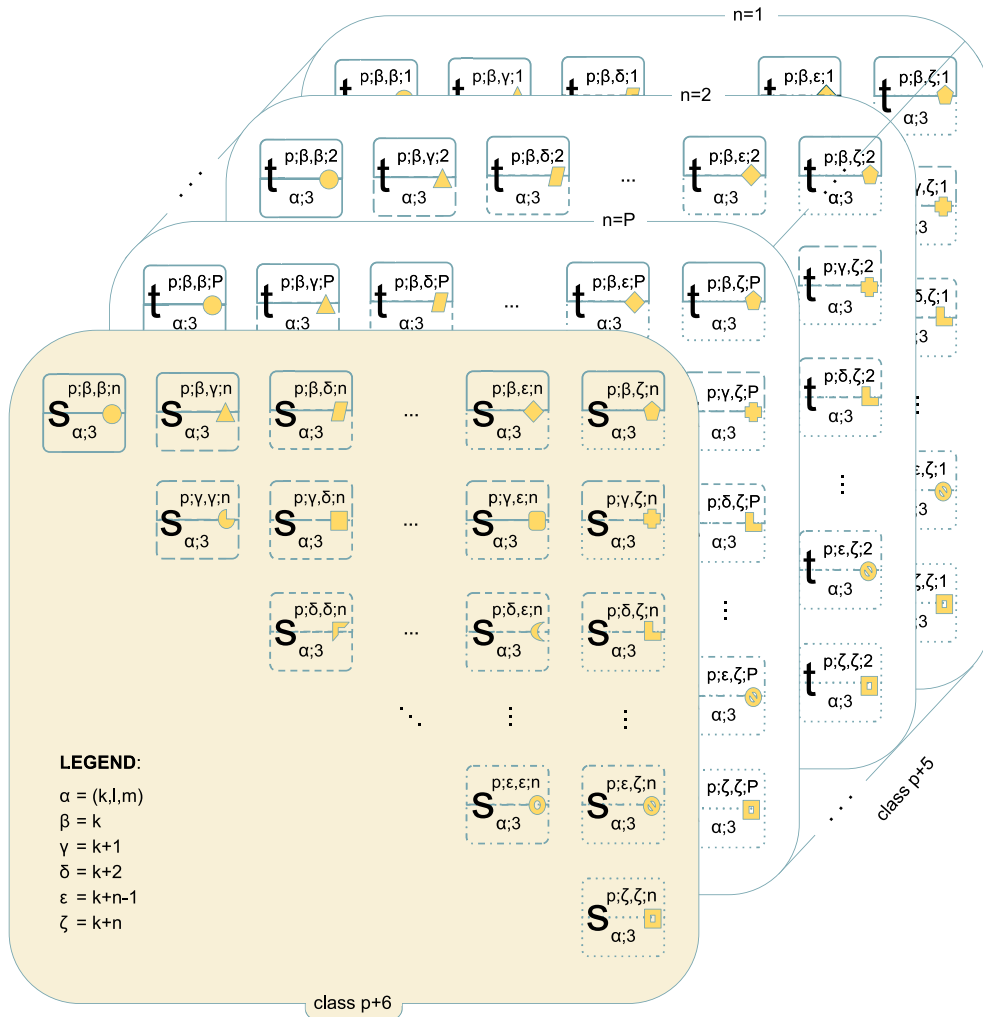


Fig. 8. Relationships between classes $p+5$ and $p+6$ for p -order functions. Each task from class $p+6$ corresponds to the approximation of the function value using Gaussian quadrature, therefore it depends on P tasks from class $p+5$. The task s depends on all tasks t with regards to its distribution in subsequent sheets $1, 2, \dots, P$.

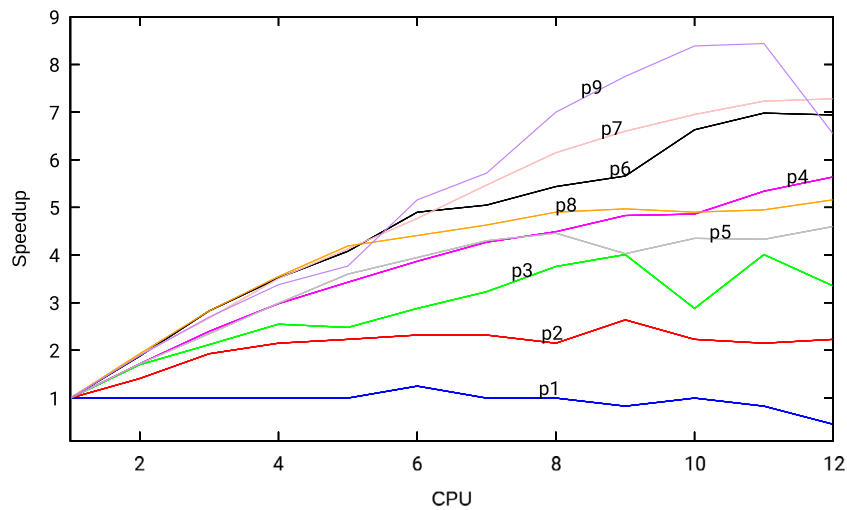


Fig. 9. Strong scaling speedup for classical integration algorithm. Computations performed on 30^3 elements mesh, different polynomial orders. Parallelism inside element.

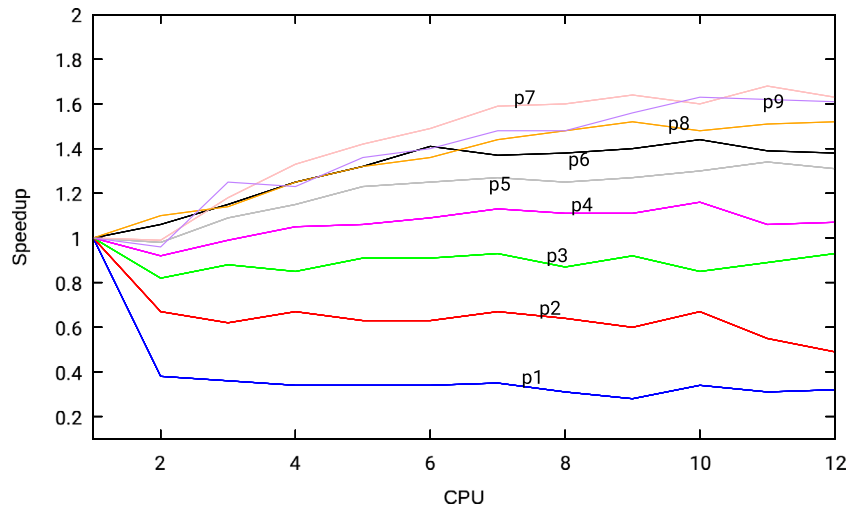


Fig. 10. Strong scaling speedup for sum factorization algorithm. Computations performed on 30^3 elements mesh, different polynomial orders. Parallelism inside element.

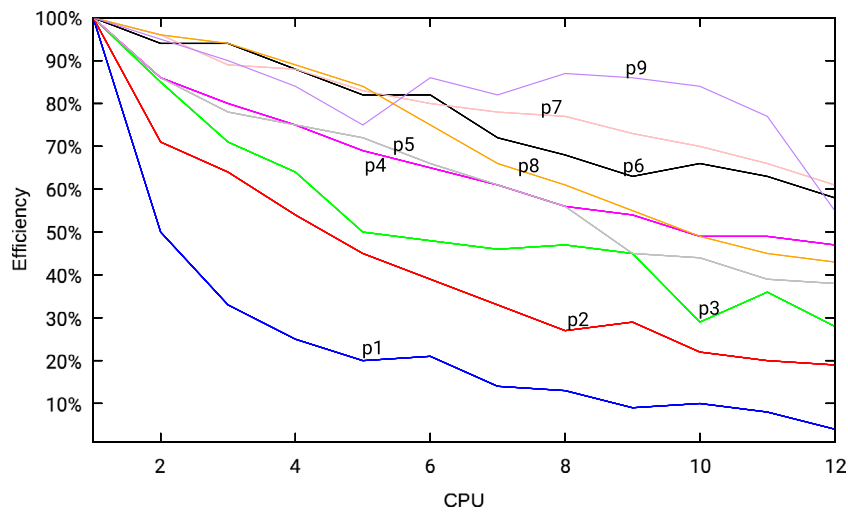


Fig. 11. Strong scaling efficiency for classical integration algorithm. Computations performed on 30^3 elements mesh, different polynomial orders. Parallelism inside element.

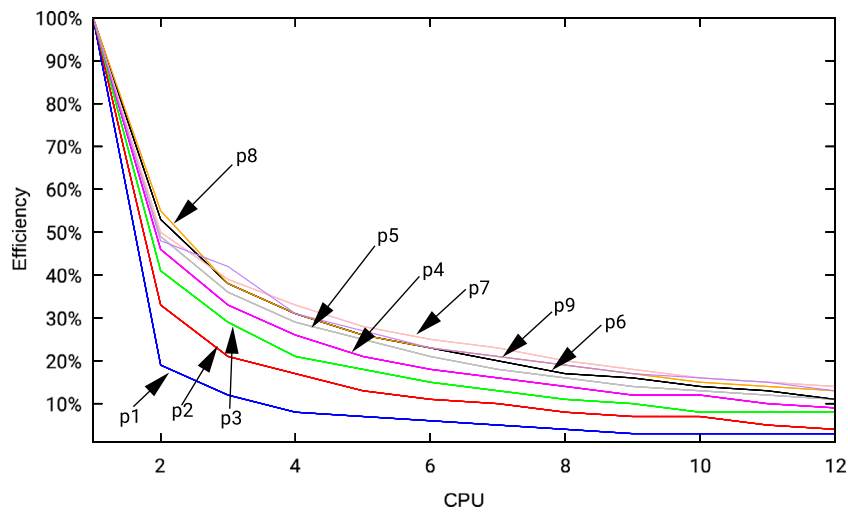


Fig. 12. Strong scaling efficiency for sum factorization algorithm. Computations performed on 30^3 elements mesh, different polynomial orders. Parallelism inside element.

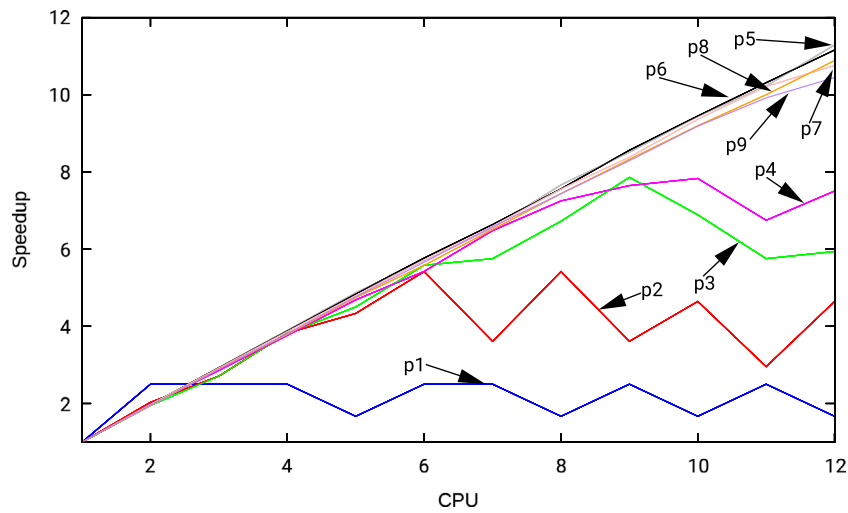


Fig. 13. Strong scaling speedup for classical integration algorithm. Computations performed on 30^3 elements mesh, different polynomial orders. Parallelism over all elements.

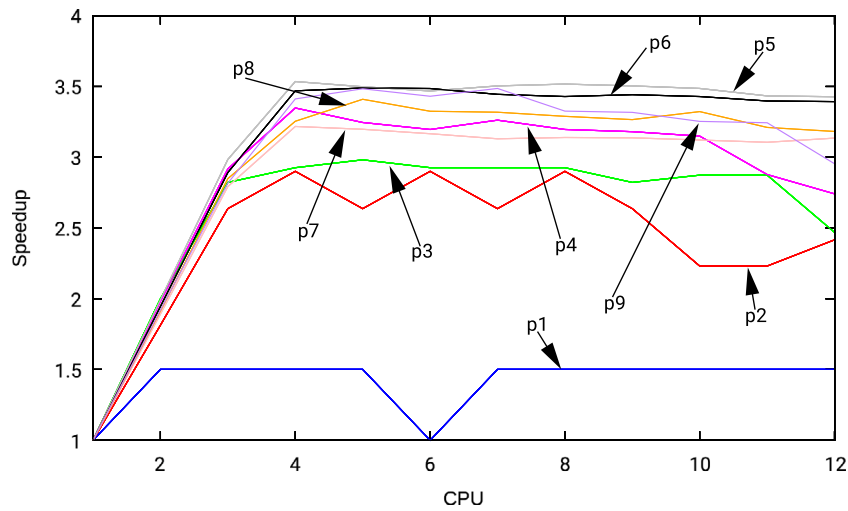


Fig. 14. Strong scaling speedup for sum factorization algorithm. Computations performed on 30^3 elements mesh, different polynomial orders. Parallelism over all elements.

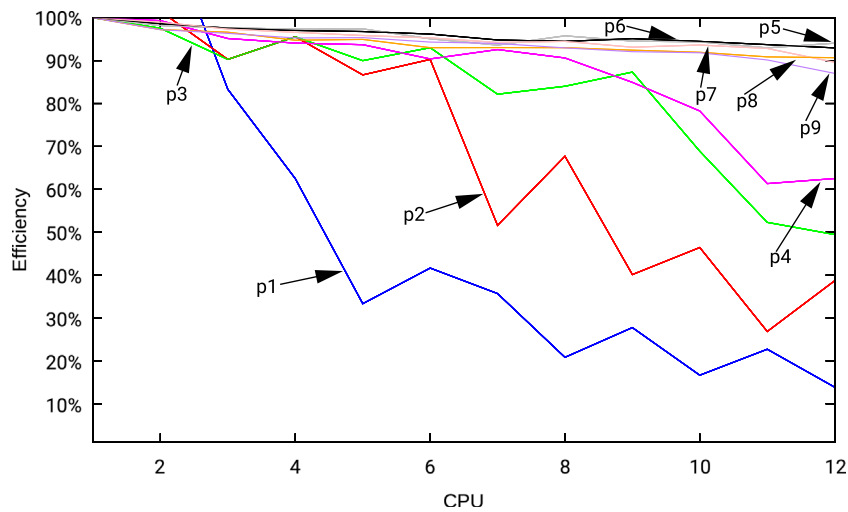


Fig. 15. Strong scaling efficiency for classical integration algorithm. Computations performed on 30^3 elements mesh, different polynomial orders. Parallelism over all elements.

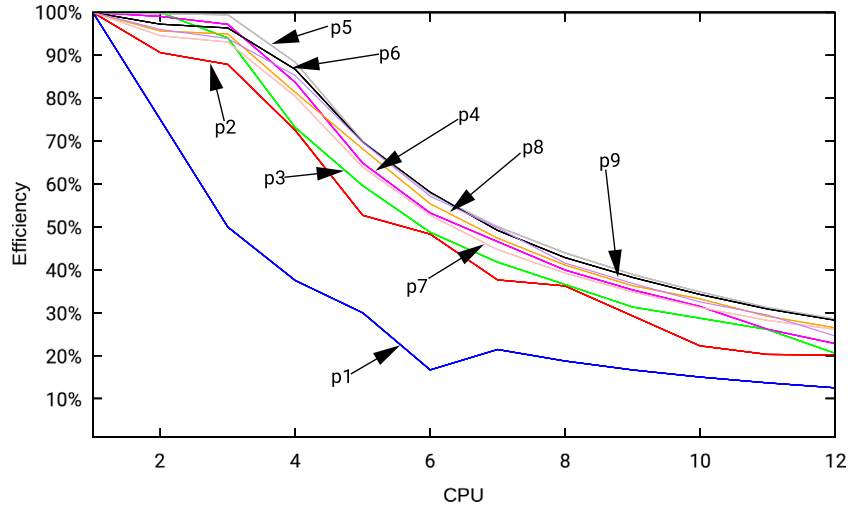


Fig. 16. Strong scaling efficiency for sum factorization algorithm. Computations performed on 30^3 elements mesh, different polynomial orders. Parallelism over all elements.

Table 6

Classical integration method. Bottom index i stands for “inside element”, e over all elements, and c combined.

p	t_{base}	v_i	$S_i(v)$	P_i	$S_i(\infty)$	v_e	$S_e(v)$	P_e	$S_e(\infty)$	$S_c(\infty)$
1	0.08	6	1.35	0.31	1.45	3	2.5	0.9	10.00	14.50
2	0.91	6	2.32	0.68	3.15	6	5.4	0.98	45.00	141.75
3	6.70	9	4.01	0.84	6.43	9	7.8	0.98	52.00	314.36
4	36.56	12	5.64	0.90	9.75	10	7.8	0.97	31.91	311.12
5	143.86	12	4.6	0.85	6.84	12	11.29	0.99	174.92	1196.45
6	562.03	11	6.98	0.94	17.36	12	11.15	0.99	144.29	1984.07
7	1622.20	12	7.28	0.94	16.97	12	10.75	0.99	94.60	1605.36
8	4586.81	12	5.16	0.88	8.30	12	10.88	0.99	106.86	886.94
9	11752.54	12	8.54	0.96	27.15	12	10.44	0.99	73.62	1998.78

Table 7

Sum factorization. Bottom index i stands for “inside element”, e over all elements, and c combined.

p	t_{base}	v_i	$S_i(v)$	P_i	$S_i(\infty)$	v_e	$S_e(v)$	P_e	$S_e(\infty)$	$S_c(\infty)$
1	0.05	1	1	0	1	2	1.5	0.67	3	3.00
2	0.29	1	1	0	1	4	2.9	0.87	7.91	7.91
3	1.38	1	1	0	1	4	2.9	0.87	7.91	7.91
4	5.12	10	1.16	0.15	1.18	4	3.3	0.93	14.14	16.69
5	15.15	11	1.34	0.28	1.39	4	3.5	0.95	21	29.19
6	40.63	10	1.44	0.34	1.51	4	3.5	0.95	21	31.71
7	109.47	11	1.68	0.45	1.80	4	3.2	0.92	12	21.60
8	202.17	9	1.52	0.38	1.63	4	3.4	0.94	17	27.71
9	394.28	10	1.63	0.43	1.75	4	3.5	0.95	21	36.75

observe outstanding speedup for classical method in both scenarios of parallelism. Furthermore Figs. 11 and 15 proven high efficiency of hardware utilization. Figures present increased parallel performance (speedup and efficiency) for higher polynomial order (p) B-spline basis functions.

Fig. 10 present unexpected behavior of sum factorization with parallel loops inside elements. Even parallel loops over all elements, presented in Fig. 14 scale up to 4 cores with expected behavior. Above four cores, speedup remains at a constant level. This corresponds with low efficiency in multicore applications, as can be seen in Figs. 12 and 16.

From Tables 6 and 7, we can observe that the theoretical maximum speedup for the classical method behaves similarly to the results presented in [10]. In Diekert graphs (Figs. 2–8), it can be observed that sum factorization requires a multitude more memory synchronizations than the classical method.

Using values from Tables 6 and 7, we also compare computational times for the classical integration and the sum factorization in several

scenarios. We focused on $p = 9$ since, theoretically, it should be the best scenario of sum factorization. We take into consideration three scenarios for a 30^3 mesh size; (1) Single-core CPU execution, (2) Shared memory CPU computations, (3) (Multiple) GPU execution. Classical integration on single-core takes 11752.54 s, 12 core OpenMP implementation takes 1125.72 s, and estimated GPU implementation should take 5.87 s. Sum factorization integration on a single core takes 394.28 s, Four-core OpenMP implementation takes 112.65 s and estimated GPU implementation should take 10.78 s.

5. Conclusions

In terms of computational performance, we discussed and compared two standard methods used for the integration in IGA-FEM; the classical integration method and sum factorization. For the comparison, we considered several scenarios of performing a shared memory layer of computations on hybrid memory clusters. First, we consider a single-core implementation as the baseline. Then, we measure experimental performance in two ways of parallel integration in shared memory, using OpenMP, with parallel loops over elements and parallel loops inside elements. In the final scenario, we estimate performance on massively parallel shared-memory machines, such as GPU, by combining maximum scalability estimates (see Section 4.3).

As expected, when assigned to a specific computational node, the sum factorization method performs better than the classical integration method. From the numerical results with a polynomial degree $p = 9$, being the worst-case scenario from the considered experiments, we can observe that the classical method is approximately 70 times slower than the sum factorization method in both scenarios of parallel integration in shared memory. Even though, when comparing single-core sum factorization with parallelized on 12 CPU cores classical integration method, still sum factorization is the clear winner.

When considering parallelized loops inside the elements, we observe very efficient parallelization for the classical integration method. However, sum factorization does not parallelize as expected. Indeed, we observe an evident loss in performance when considering more than one core. Additionally, when considering the standard loops over elements, we observe performance gain for sum factorization only up to 4 cores in a shared memory (see Figs. 10, 14).

Finally, based on the previous work [10], we can estimate the performance for two-layer parallelization technique on massively parallel machines, such as GPUs. In such a case, the classical integration method parallelizes outstandingly, resulting in faster execution than sum factorization. In other words, numerical results show that the classical integration method running on a GPU can be faster than sum

factorization by one or two orders of magnitude. A possible explanation for this small performance gain, or lack of such in some cases for sum factorization, is possibly limited by the memory synchronization and the memory access. Despite the higher computational cost of the classical method concerning sum factorization, such a method requires fewer data dependencies and synchronizations than sum factorization. However, when considering low cores machines, sum factorization is the method of choice over the classical one. The best parallelization strategy we observe in such a case is to use 4 CPU cores in shared memory.

Future works in this direction will be directed to obtain a fully optimal methodology of parallel computations involving all parts of IGA-FEM. Therefore, we will apply trace theory to other scenarios related to numerical computations, for instance, in the resolution of the system of equations.

CRedit authorship contribution statement

Maciej Woźniak: Supervision, Methodology, Software, Visualization, Writing – original draft. **Anna Szyszka:** Software, Visualization, Writing – original draft. **Sergio Rojas:** Writing – review & editing, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This project has received funding from the European Union's Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant agreement No 777778 (MATHROCKS). The work of SR has also been partially supported by the Chilean grant ANID Fondecyt No 3210009.

References

- [1] G. Strang, G.J. Fix, D. Griffin, *An Analysis of the Finite-Element Method*, Wellesley-Cambridge Press, 1973.
- [2] T.J. Hughes, *the Finite Element Method: Linear Static and Dynamic Finite Element Analysis*, Courier Corporation, 2012.
- [3] P.G. Ciarlet, *the Finite Element Method for Elliptic Problems*, Society for Industrial and Applied Mathematics, 2002.
- [4] J.A. Cottrell, T.J.R. Hughes, Y. Bazilevs, *Isogeometric Analysis: Toward Integration of CAD and FEA*, John Wiley & Sons, Ltd., 2009.
- [5] C. de Boor, Subroutine package for calculating with B-splines, *SIAM J. Numer. Anal.* 14 (3) (1971) 441–472.
- [6] M. Łoś, A. Khusek, M.A. Hassaan, K. Pingali, W. Dzwiniel, M. Paszyński, Parallel fast isogeometric L2 projection solver with GALOIS system for 3D tumor growth simulations, *Comput. Methods Appl. Mech. Engrg.* 343 (2019) 1–22.
- [7] V. Puzyrev, M. Łoś, G. Gurgul, V.M. Calo, W. Dzwiniel, M. Paszyński, Parallel splitting solvers for the isogeometric analysis of the Cahn-Hilliard equation, *Comput. Methods Biomech. Biomed. Eng.* 22 (2019) 1269–1281.
- [8] M. Woźniak, M. Smółka, A. Cortes, M. Paszyński, R. Schaefer, Scalability of direct solver for non-stationary Cahn-Hilliard simulations with linearized time integration scheme, *Procedia Comput. Sci.* 80 (2016) 834–844.
- [9] N. Collier, D. Pardo, L. Dalcin, M. Paszyński, V. Calo, The cost of continuity: A study of the performance of isogeometric finite elements using direct solvers, *Comput. Methods Appl. Mech. Engrg.* 213–216 (2012) 353–361.
- [10] A. Szyszka, M. Woźniak, R. Schaefer, Concurrent algorithm for integrating three-dimensional B-spline functions into machines with shared memory such as GPU, *Comput. Methods Appl. Mech. Eng.* 398 (2022) 115201, <http://dx.doi.org/10.1016/j.cma.2022.115201>.
- [11] CUDA Toolkit Documentation, <https://docs.nvidia.com/cuda>.
- [12] K. Banaś, F. Kružel, J. Bielański, Optimal kernel design for finite-element numerical integration on GPUs, *Comput. Sci. Eng.* 22 (6) (2020) 61–74, <http://dx.doi.org/10.1109/MCSE.2019.2940656>.

- [13] K. Banaś, F. Kružel, J. Bielański, Finite element numerical integration for first order approximations on multi- and many-core architectures, *Comput. Methods Appl. Mech. Engrg.* 305 (2016) 827–848, <http://dx.doi.org/10.1016/j.cma.2016.03.038>, URL <https://www.sciencedirect.com/science/article/pii/S0045782516301256>.
- [14] A. Karatarakis, P. Karakitsios, M. Papadarakakis, GPU accelerated computation of the isogeometric analysis stiffness matrix, *Comput. Methods Appl. Mech. Engrg.* 269 (2014) 334–355, <http://dx.doi.org/10.1016/j.cma.2013.11.008>, URL <https://www.sciencedirect.com/science/article/pii/S0045782513003034>.
- [15] Cyfronet KDM, https://kdm.cyfronet.pl/portal/Main_page.
- [16] Stampede2 User Guide, <https://portal.tacc.utexas.edu/user-guides/stampede2>.
- [17] Summit, Oak Ridge National Laboratory, <https://www.olcf.ornl.gov/summit/>.
- [18] CUDA Toolkit Documentation, programming guide, <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>.
- [19] R.R. Hiemstra, F. Calabrò, D. Schillinger, T.J. Hughes, Optimal and reduced quadrature rules for tensor product and hierarchically refined splines in isogeometric analysis, *Comput. Methods Appl. Mech. Engrg.* 316 (2017) 966–1004.
- [20] S.A. Orszag, *Spectral Methods for Problems in Complex Geometries*, in: *Numerical Methods for Partial Differential Equations*, Academic Press, 1979, pp. 273–305, <http://dx.doi.org/10.1016/B978-0-12-546050-7.50014-9>.
- [21] L. Demkowicz, J. Kurtz, D. Pardo, M. Paszyński, a.A.Z. Waldemar Rachowicz, *Computing with hp-Adaptive Finite Elements, Vol. II. Frontiers: Three Dimensional Elliptic and Maxwell Problems with Applications*, Chapman and Hall/CRC, 2007.
- [22] P.E. Vos, S.J. Sherwin, R.M. Kirby, From h to p efficiently: Implementing finite and spectral/hp element methods to achieve optimal performance for low- and high-order discretisations, *J. Comput. Phys.* 229 (13) (2010) 5161–5181, <http://dx.doi.org/10.1016/j.jcp.2010.03.031>.
- [23] M. Ainsworth, G. Andriamaro, O. Davydov, Bernstein-Bézier finite elements of arbitrary order and optimal assembly procedures, *SIAM J. Sci. Comput.* 33 (6) (2011) 3087–3109, <http://dx.doi.org/10.1137/11082539X>.
- [24] G.E. Karniadakis, G. Karniadakis, S. Sherwin, *Spectral/Hp Element Methods for Computational Fluid Dynamics*, Oxford University Press on Demand, 2005.
- [25] T. Eibner, *Fast Algorithms for Setting Up the Stiffness Matrix in hp-FEM: A Comparison*, The University of Reading, Numerical Analysis Report, Citeseer, 2005.
- [26] P. Antolin, A. Buffa, F. Calabrò, M. Martinelli, G. Sangalli, Efficient matrix computation for tensor-product isogeometric analysis: The use of sum factorization, *Comput. Methods Appl. Mech. Engrg.* 285 (2015) 817–828, <http://dx.doi.org/10.1016/j.cma.2014.12.013>.
- [27] A. Bressan, S. Takacs, Sum factorization techniques in isogeometric analysis, *Comput. Methods Appl. Mech. Engrg.* 352 (2019) 437–460, <http://dx.doi.org/10.1016/j.cma.2019.04.031>.
- [28] A. Mazurkiewicz, Concurrent program schemes and their interpretations, *DAIMI Rep. Ser.* 78 (1977).
- [29] C.A. Petri, *Kommunikation mit Automaten* (Ph.D. thesis), Universität Hamburg, 1962, <http://edoc.sub.uni-hamburg.de/informatik/volltexte/2011/160/>.
- [30] J.C.M. Baeten, A brief history of process algebra, *Theoret. Comput. Sci.* 335 (2004) 131–146.
- [31] H. Baker, C. Hewitt, Laws for communicating parallel processes, in: *IFIP Congress*, 1977, pp. 987–992.
- [32] V. Diekert, G. Rozenberg, *The Book of Traces*, World Scientific, 1995.
- [33] C. de Boor, *A Practical Guide to Splines*, in: *Mathematics of Computation*, vol. 17, Springer-Verlag, 1978.
- [34] L. Schumaker, *Spline Functions: Basic Theory*, Cambridge University Press, 2007.
- [35] R.R. Hiemstra, G. Sangalli, M. Tani, F. Calabrò, T.J. Hughes, Fast formation and assembly of finite element matrices with application to isogeometric linear elasticity, *Comput. Methods Appl. Mech. Engrg.* 355 (2019) 234–260, <http://dx.doi.org/10.1016/j.cma.2019.06.020>.
- [36] V. Diekert, Y. Métivier, Partial commutation and traces, in: *Handbook of Formal Languages*, Springer, 1997, pp. 457–533.



Maciej Woźniak is an assistant professor on Institute of Computer Science in AGH University of Science and Technology, Kraków, Poland. He received his master degree in Computer Science in 2013. He received his bachelor degree in Business Management in 2015. In 2017 he received his Ph.D. in computational science from AGH-UST. Since 2012 he is a member of Prof. Maciej Paszyński research group, working primarily on fast parallel direct solvers for isogeometric finite element methods targeting different parallel architectures.



Anna Szyszka graduated in 2020 with Master degree at Computer Science at AGH University, Krakow, Poland. Her research interest includes optimization of algorithms for different architectures of parallel machines, isogeometric analysis as well as formal models of concurrency.



Sergio Rojas is a professor of mathematics at Pontificia Universidad Católica de Valparaíso. He got his M.Sc. and Ph.D. in Engineering Sciences from Pontificia Universidad Católica de Chile in 2017, his M.Sc. in Mathematics from Pavia University in 2011, and his B.S. in Mathematics in 2008 from Pontificia Universidad Católica de Valparaíso. His research interests include numerical analysis, mathematical modeling, and scientific computing.